

INSIGHT ENGINE

Autonomous Content-to-Action Agent System
Software Requirements Specification — v1.0

Google Antigravity Hackathon · Challenge 1
Saad · Omer · Moeez
Date: May 16, 2026

Saad Backend Orchestration	· Antigravity	Omer Frontend · Web Dashboard	Moeez Mobile App · Simulation Layer
---	-------------------	---	---

1. PROJECT OVERVIEW

1.1 Project Name & Tagline

Insight Engine — An agentic AI system that ingests unstructured business content, extracts non-trivial insights, reasons about real-world impact, and autonomously executes actions — all orchestrated through Google Antigravity.

1.2 Problem Statement

Organizations receive enormous volumes of unstructured signals — sales reports, news articles, supplier emails, operational dashboards — but existing AI tools stop at summarization. Insight Engine bridges the gap between analysis and action: it reasons across signals, identifies causal relationships, decides on the highest-priority action, and simulates its execution with a visible state change.

1.3 Domain: Supply Chain & Business Operations

The chosen domain is logistics and supply chain operations for a Pakistani importer/distributor. This domain was selected because:

- Unstructured input is natural: sales CSVs, news articles about fuel/forex, vendor emails
- Actions are visually distinct and compelling: pricing updates, campaigns, reorder triggers
- Causal reasoning is non-trivial: multiple signals interact (e.g. fuel cost + regional order decline = compounding margin compression, not independent events)
- Highly relatable for Pakistani context, strengthening demo credibility with judges

1.4 Evaluation Scoring Strategy

Criterion	Weight	How We Score High
Google Antigravity Usage	25%	5-agent pipeline fully orchestrated in Antigravity. Workplan, tasks, reasoning steps
Agentic Reasoning & Workflow	20%	Multi-step planner with explicit decision rationale. Not just suggestions — the system
Insight & Decision Quality	20%	Causal chain reasoning across 2+ signals. Analyst identifies compounding effects
Action Simulation & Outcome	15%	3 visible simulated actions: pricing table diff, campaign card creation, WhatsApp n
Technical Implementation	10%	FastAPI + Supabase + Antigravity. Clean separation. Real DB writes, not fake mo
Innovation & UX	10%	Pre-loaded demo scenarios. Mobile + Web. Before/after state visualized live.

2. SYSTEM ARCHITECTURE

2.1 High-Level Architecture Diagram



2.2 Agent Descriptions

INGESTOR AGENT

- Accepts input: raw text, PDF file, URL (news article), or CSV (sales data)
- Normalizes all formats into a structured SignalObject: {source, type, raw_text, timestamp}
- Runs basic entity extraction: numbers, dates, organizations, locations
- Passes normalized signal array to Analyst Agent
- Handles edge cases: empty input, malformed CSV, unreachable URLs

ANALYST AGENT

- Receives array of SignalObjects from Ingestor
- Identifies causal relationships between signals (not single-point summaries)

- Example reasoning: 'Fuel +18% AND regional orders -25% = compounding margin compression in southern routes'
- Scores each insight by: novelty, severity, and actionability
- Outputs InsightReport: {primary_insight, causal_chain, severity_score, affected_domains}

PLANNER AGENT

- Receives InsightReport from Analyst
- Generates 2–3 candidate actions ranked by urgency × feasibility × impact
- Selects the top-priority action and commits to it (not just a suggestion list)
- Outputs ActionPlan: {selected_action, reasoning, parameters, fallback_actions}
- Reasoning must be traceable — logged in Antigravity workplan

EXECUTOR AGENT

- Receives ActionPlan from Planner
- Executes 1 primary + up to 2 secondary simulated actions via FastAPI calls
- Primary action examples: pricing table update, campaign creation, reorder trigger
- Writes before/after state to Supabase (products, campaigns, notifications tables)
- Returns ExecutionLog: {action_taken, before_state, after_state, timestamp, status}

REPORTER AGENT

- Receives ExecutionLog + InsightReport
- Generates final structured output: insight → impact → action → result
- Formats output for both Web (JSON) and Mobile (condensed card format)
- Calculates projected impact metrics (e.g. 'Projected reach: 5,000 users')
- Produces agent trace summary for judge review

3. DATA FLOW & EXAMPLE SCENARIO

3.1 End-to-End Flow

Step	Agent	Input	Output
1	Ingestor	Sales CSV + News article URL	SignalObject array: [{orders_declined_25pct_lahore}, {fuel_up_18pct_effective_immediately}, {supplier_lead_time_extended_3weeks}]
2	Analyst	SignalObject array	InsightReport: 'Margin compression in south compounding — revenue gap widening due to falling demand + rising costs'
3	Planner	InsightReport	ActionPlan: Launch regional discount + update delivery pricing [South, 15% off, 2 weeks]
4	Executor	ActionPlan	Supabase writes: pricing row updated, campaign row inserted, notification sent
5	Reporter	ExecutionLog + InsightReport	Final output card: insight → impact → actions → projected reach

3.2 Concrete Demo Scenario (Pre-loaded for Judges)

The following scenario will be pre-loaded as Demo Scenario 1 — press one button and the full pipeline runs:

INPUT SIGNALS	
Source 1	sales_report_q2.csv — Orders in Lahore region down 25% vs Q1
Source 2	https://dawn.com/fuel-price-hike — Fuel up 18% effective immediately
Source 3	vendor_email.txt — Supplier lead times extended by 3 weeks
ANALYST OUTPUT	
Primary Insight	Three compounding signals: falling regional demand + rising delivery cost + supply delay = immediate revenue impact
Severity	HIGH (score: 8.7/10)
Causal Chain	Fuel ↑ → delivery cost ↑ → margin ↓ while orders ↓ → revenue gap widens
PLANNER DECISION	
Selected Action	Action 1: Launch regional discount campaign (Lahore, 15% off, 2 weeks)
Reasoning	Demand recovery has higher ROI than cost-cutting given supplier delay is temporary
Fallback	Action 2: Update delivery pricing structure if campaign ROI < 20% in 72h
EXECUTOR SIMULATIONS	
Sim 1	Supabase: campaigns table → row inserted {region: Lahore, discount: 15%, status: active}
Sim 2	Supabase: pricing table → delivery_cost_lahore updated from 250 → 295 PKR

Sim 3	Notification drafted: WhatsApp message template for 5,000 Lahore customers
REPORTER OUTPUT	
Projected Impact	Revenue recovery: +PKR 1.2M over 14 days Reach: 5,000 users
Before State	No active campaign Old pricing No supplier alert
After State	Campaign LIVE Pricing updated Notification queued

4. DATABASE SCHEMA (SUPABASE)

All action simulations write to real Supabase tables. This ensures the before/after state is genuinely real, not a mock object.

Table: signals

Column	Type	Notes
id	uuid	PK, auto
source_type	text	pdf url csv text
raw_content	text	Original input
normalized_json	jsonb	Parsed SignalObject
created_at	timestampz	Auto

Table: insight_reports

Column	Type	Notes
id	uuid	PK, auto
signal_ids	uuid[]	FK → signals
primary_insight	text	Main finding
causal_chain	text	Reasoning chain
severity_score	float4	0.0–10.0
affected_domains	text[]	e.g. [pricing, logistics]
created_at	timestampz	Auto

Table: action_plans

Column	Type	Notes
id	uuid	PK, auto
insight_id	uuid	FK → insight_reports
selected_action	text	What Planner decided
reasoning	text	Why this action
parameters	jsonb	Action-specific params
fallback_actions	jsonb	Ranked alternatives
created_at	timestampz	Auto

Table: campaigns

Column	Type	Notes
id	uuid	PK, auto
plan_id	uuid	FK → action_plans
name	text	Campaign name
region	text	Target region
discount_pct	float4	Discount amount
status	text	draft active completed
projected_reach	int4	Est. users reached
created_at	timestampz	Auto

Table: pricing_log

Column	Type	Notes
id	uuid	PK, auto
plan_id	uuid	FK → action_plans
item_name	text	e.g. delivery_cost_lahore
before_value	float4	Previous value
after_value	float4	Updated value
changed_at	timestampz	Auto

Table: notifications

Column	Type	Notes
id	uuid	PK, auto
plan_id	uuid	FK → action_plans
channel	text	whatsapp email sms
recipient_count	int4	Target audience size
message_body	text	Drafted message
status	text	drafted sent failed
created_at	timestampz	Auto

Table: execution_logs

Column	Type	Notes
id	uuid	PK, auto
plan_id	uuid	FK → action_plans

actions_taken	jsonb	Array of completed actions
before_snapshot	jsonb	System state before
after_snapshot	jsonb	System state after
agent_trace	jsonb	Full Antigravity trace
created_at	timestampz	Auto

5. FASTAPI ENDPOINT DESIGN

All endpoints are hosted on Railway via FastAPI. Antigravity calls these endpoints as tools during agent execution.

Method	Endpoint	Called By	Description
POST	/ingest	Antigravity Ingestor Agent	Accepts multipart: text, file upload, or URL. Returns SignalObject
POST	/analyze	Antigravity Analyst Agent	Accepts SignalObject array. Returns InsightReport with causal graph
POST	/plan	Antigravity Planner Agent	Accepts InsightReport. Returns ranked ActionPlan with selected actions
POST	/execute	Antigravity Executor Agent	Accepts ActionPlan. Writes to Supabase. Returns ExecutionLog
POST	/report	Antigravity Reporter Agent	Accepts ExecutionLog + InsightReport. Returns formatted final report
GET	/state/before	Frontend / Mobile	Returns current DB state (pricing, campaigns, notifications)
GET	/state/after/{plan_id}	Frontend / Mobile	Returns DB state after a specific execution. Powers before/after comparison
GET	/logs/{plan_id}	Frontend / Mobile	Returns full agent trace and execution log for a given run.
GET	/scenarios	Frontend / Mobile	Returns list of pre-loaded demo scenarios for judge demo.
POST	/run-scenario/{id}	Frontend / Mobile	Triggers a full end-to-end pipeline run for a pre-loaded scenario

6. TECH STACK

Layer	Technology	Purpose	Owner
Agent Orchestration	Google Antigravity	Core agent workflows, planning, tool execution	Saad
Backend Framework	FastAPI (Python)	REST API, agent tool endpoints, business logic	Saad
Database	Supabase (Postgres)	All state persistence — signals, actions, logs	Saad
Hosting (Backend)	Railway	FastAPI deployment, zero-cost tier	Saad
Web Frontend	React + Vite + TailwindCSS	Dashboard: input, pipeline trace, before/after	Omer
Real-time Updates	Supabase Realtime	Live state change without polling	Omer
Web Hosting	Vercel	React app deployment	Omer
Mobile App	Flutter	Cross-platform mobile prototype	Moez
File Parsing	PyMuPDF / pdfplumber	PDF text extraction in Ingestor agent	Saad
HTTP Client	httpx (async)	Antigravity → FastAPI tool calls	Saad
State Management (Web)	Zustand	Frontend state for pipeline steps	Omer
Demo Scenarios	JSON seed file	Pre-loaded inputs for judge demo	Moez

7. DEVELOPER TASK BREAKDOWN

Each developer owns a complete vertical slice. Tasks are sequenced so that parallel work is possible from Day 1, with integration points clearly defined.

SAAD

Backend Lead · Antigravity Orchestration · FastAPI · Supabase

Primary Responsibility

Saad owns the entire intelligence layer — all 5 Antigravity agents, the FastAPI backend, and the Supabase schema. Everything that makes the system 'think and act' lives in Saad's domain.

Priority	Task	Detail	Deliverable
P0	Supabase Schema	Create all 7 tables from Section 4. Enable Realtime on Live Supabase project with notifications.	Live Supabase schema
P0	Antigravity — Ingestor Agent	Build agent that accepts text/PDF/URL/CSV input. Call POST /ingest to agent. Return Signal Object and Signal Array.	POST /ingest endpoint. Return Signal Object and Signal Array.
P0	FastAPI — /ingest endpoint	Parse text, extract PDF with pdfplumber, fetch URL content. Ingest into Signal Object. Write to signal array.	POST /ingest endpoint. Write to signal array.
P1	Antigravity — Analyst Agent	Receives signal array. Reasons across signals for causal chains. Assigns severity. Call POST /analyze endpoint.	POST /analyze endpoint. Return InsightReport.
P1	FastAPI — /analyze endpoint	Call (Gemini via Antigravity) to reason across signals. Return InsightReport.	POST /analyze endpoint. Return InsightReport.
P1	Antigravity — Planner Agent	Receives InsightReport. Generates 2-3 candidate actions. Rank by severity. Call POST /plan endpoint.	POST /plan endpoint. Return ActionPlan.
P1	FastAPI — /plan endpoint	Generate ActionPlan with ranking. Commit to one selected action. Return ActionPlan.	POST /plan endpoint. Return ActionPlan.
P2	Antigravity — Executor Agent	Receives ActionPlan. Calls /execute. Handles 3 simultaneous campaigns, pricing, notifications.	POST /execute endpoint. Return Campaigns.
P2	FastAPI — /execute endpoint	Write campaign row to campaigns. Write pricing diff to pricing. Write notification to notifications.	POST /execute endpoint. Return Campaigns.
P2	Antigravity — Reporter Agent	Generates final report card. Calls POST /report. Return ReportCard to frontend.	POST /report endpoint. Return ReportCard.
P2	FastAPI — /report, /state endpoints	Return ReportCard. Agent trace retrieval. Scale up/down endpoints live.	POST /report endpoint. Return ReportCard.
P3	Pre-loaded Demo Scenarios	Seed 3 scenarios in DB. Endpoint /run-scenario/{id} triggers full pipeline for each.	POST /run-scenario/{id} endpoint.
P3	Railway Deployment	Deploy FastAPI to Railway. Set env vars. Share base URL with client.	Live API URL.

OMER

Frontend Lead · React Web Dashboard · UX · Before/After Visualization

Primary Responsibility

Omer owns the web dashboard — the primary interface judges will interact with on desktop. The dashboard must visualize the pipeline in real-time, show before/after state changes, and display agent traces clearly.

Priority	Task	Detail	Deliverable
P0	Project Setup	Vite + React + TailwindCSS. Supabase JS client. Zustand + Vercel Deployment	Live Vercel Deployment
P0	Input Panel	3 input modes: text area, PDF file upload, URL paste. Input validation and error handling	Prepared with 3 modes and error handling
P1	Pipeline Trace Visualization	5-step progress indicator: Ingestor → Analyst → Planner → Action → Review. Each step shows status and duration	Real-time pipeline tracker
P1	Insight Report Card	Display InsightReport: primary insight (large text), causal chain (arrow diagram), severity score (color-coded)	Insight card (arrow diagram)
P1	Action Plan Card	Display selected action with reasoning. Show fallback action if primary is not committed	Action plan collapse/expand
P2	Before/After State Panel	Side-by-side table diff: show pricing_log before vs after. Highlight changes in green/red	Before/After diff table
P2	Supabase Realtime Integration	Subscribe to campaigns, pricing_log, notifications table. Real-time updates without refresh	Real-time updates
P2	Agent Trace Panel	Collapsible section: show full JSON trace from Antigravity. Syntax highlighting. Available for debugging	Agent trace panel
P3	Projected Impact Dashboard	Reporter output displayed as metric cards: Projected Revenue, Estimated Reach, Actions Taken	Revenue, Reach, Actions
P3	Responsive Polish	Ensure dashboard works on 1280px+ screens. Dark mode toggle button	Dark mode toggle
P3	Demo Recording Support	Add a 'Reset' button that clears state and re-runs. Ensure pipeline animation is smooth for screen recording	Reset button

MOEEZ

Mobile Lead · Flutter App · Demo Scenarios · Action Simulation Layer

Primary Responsibility

Moeez owns the Flutter mobile app — the MANDATORY deliverable per the challenge rules — and the demo scenario content. The mobile app is a condensed version of the pipeline with a focus on the action simulation output, optimized for a phone screen.

Priority	Task	Detail	Deliverable
P0	Flutter Project Setup	New Flutter project. Add deps: http, supabase_flutter, Flutter project, provider, Supabase client	Flutter project, provider, Supabase client
P0	Input Screen	Clean single screen: text input field + 3 pre-loaded scenarios buttons (labeled 'Regional Sales Drop', 'Supply Chain Disruption', 'Economic Recession')	Input screen with 3 buttons
P1	Pipeline Status Screen	Vertical stepper: 5 steps with icons. Each step fills in with results when API polling confirms completion	Stepper screen with API polling
P1	Insight Card Screen	Full-screen card: insight headline (large bold), severity insight, causal chain in plain English. Swipe to see next	Insight screen
P1	Action Card Screen	Show selected action name, Planner's reasoning in 2-3 sentences, and fallback options as small chips	Action screen
P2	Simulation Results Screen	3 animated result tiles: (1) Pricing tile — shows old vs new value, (2) Campaign tile — shows reach and engagement	Simulation results screen with 3 tiles
P2	Impact Summary Screen	Final screen: 4 metric cards — Projected Revenue, Reach, Actions Done, Execution Time. Share button	Impact summary screen
P2	Demo Scenario Content	Write the 3 pre-loaded scenario seed data: signal input, expected insights, action params. Share as JSON files	3 scenario JSON files
P3	Notification Preview Widget	Custom Flutter widget that renders a WhatsApp-style WhatsApp bubble with drafted campaign message	WhatsApp bubble widget
P3	Offline Graceful Handling	If API unreachable, show cached last result from Supabase and list of error state with retry button.	Error handling screen
P3	App Polish for Demo Video	Add smooth page transitions (slide). Ensure no loading screen. Test on ready app, physical device for demo	Smooth transitions, no loading screen

8. INTEGRATION POINTS & HANDOFFS

These are the critical sync points between developers. Block nothing — share early, integrate often.

Handoff	From	To	What's Shared	When
API Base URL	Saad	Omer + Moeez	Railway deployment URL + Supabase credentials	Day 1 — as soon as Railway is deployed
Supabase Keys	Saad	Omer + Moeez	project URL + anon key (read-only for real time)	Day 1 — immediately after schema is ready
API Response Schemas	Saad	Omer + Moeez	JSON shapes for all endpoints (typed interfaces)	Day 1 — before Omer/Moeez start building
Scenario JSON Files	Moeez	Saad	3 scenario seed files for /scenarios DB seeding	Day 2 — Saad seeds DB, Moeez tests
Pipeline Completion Signal	Saad (API)	Omer + Moeez	GET /logs/{plan_id} returns status: complete	Ongoing — Omer polls this, Moeez tests
Before/After Snapshot Format	Saad	Omer	JSON format of /state/before and /state/after	Day 2 — before Omer builds diff page
Demo Video Coordination	All	All	Agree on scenario order, who controls each agent	Final day — rehearse once before recording

9. FINAL DELIVERABLES CHECKLIST

#	Deliverable	Owner	Criterion Served
1	Working Mobile App (Flutter) — MANDATORY	Moeez	Innovation & UX (10%)
2	Working Web App (React Dashboard)	Omer	Innovation & UX (10%)
3	Demo Video 3–5 min: input → insight → action → simulation (Moeez records, Saad narrates)	All (Moeez records, Saad narrates)	Arrival — most visible deliverable
4	Agent Trace / Logs from Antigravity (Workplan + Task Plan + Decision Flow)	Saad + Omer	Antigravity Usage (25%)
5	README with architecture overview, tools, Antigravity usage (writes), Omer + Moeez (review)	Saad (writes), Omer + Moeez (review)	Technical Implementation (10%)
6	3 Pre-loaded Demo Scenarios (one-click end-to-end)	Moeez (content) + Saad (seeding)	Action Simulation (15%)
7	Before/After State Visualization in Web Dashboard	Omer	Action Simulation (15%)
8	Causal Chain Reasoning in Insight Card (non-trivial insight) (Saad's Analyst agent)	Saad	Insight Quality (20%)

10. ANTIGRAVITY USAGE NOTES (FOR README)

How Antigravity is Central (Not Superficial)

- ✓ Antigravity orchestrates all 5 agents — it is not a wrapper around a single LLM call
- ✓ Each agent has an explicit role, tools, and reasoning instructions defined in Antigravity
- ✓ The Planner agent demonstrates autonomous decision-making: it selects ONE action and commits, not just lists options
- ✓ Antigravity's workplan and task logs form the 'Agent Trace' deliverable — automatically generated

- ✓ Tool calls to FastAPI endpoints are registered in Antigravity as named tools (ingest_content, analyze_signals, create_action_plan, execute_actions, generate_report)
- ✓ Multi-step reasoning is visible: each agent's output feeds the next as a structured input, logged by Antigravity
- ✓ The Analyst agent uses Antigravity's reasoning layer to perform cross-signal causal inference — not just keyword extraction

What NOT to Do (Common Mistakes to Avoid)

- ✗ Do NOT use Antigravity only once as a wrapper — judges will catch this immediately
- ✗ Do NOT produce generic summaries — insights must show causal reasoning (X causes Y which compounds Z)
- ✗ Do NOT simulate actions as print statements — all simulations must write to Supabase
- ✗ Do NOT show a list of suggestions — Planner must commit to one decision with explicit reasoning
- ✗ Do NOT skip the before/after state — this is required for the 15% simulation score

INSIGHT ENGINE

Build it. Ship it. Win it.

Google Antigravity Hackathon · Challenge 1 · Team: Saad · Omer · Moez